第7章作业　7.1　7.3

第7章　图

7.1 基本术语

7.2 存储结构

7.3 图的遍历

7.4 图的其他运算

7.5 图的应用

7.3 图的遍历

遍历定义：从已给的连通图中某一顶点出发，沿着一些边，访遍图中所有的顶点，且使每个顶点仅被访问一次，就叫做图的遍历，它是图的基本运算。

遍历实质：找每个顶点的邻接点的过程。

图的特点：图中可能存在回路，且图的任一顶点都可能与其它顶点相通，在访问完某个顶点之后可能会沿着某些边又回到了曾经访问过的顶点。

怎样避免重复访问?

解决思路：可设置一个辅助数组 *visited* [n]，用来标记每个被访问过的顶点。它的初始状态为0，在图的遍历过程中，一旦某一个顶点*i* 被访问，就立即改 *visited* [*i*]为1，防止它被多次访问。

图常用的遍历：

一、深度优先搜索

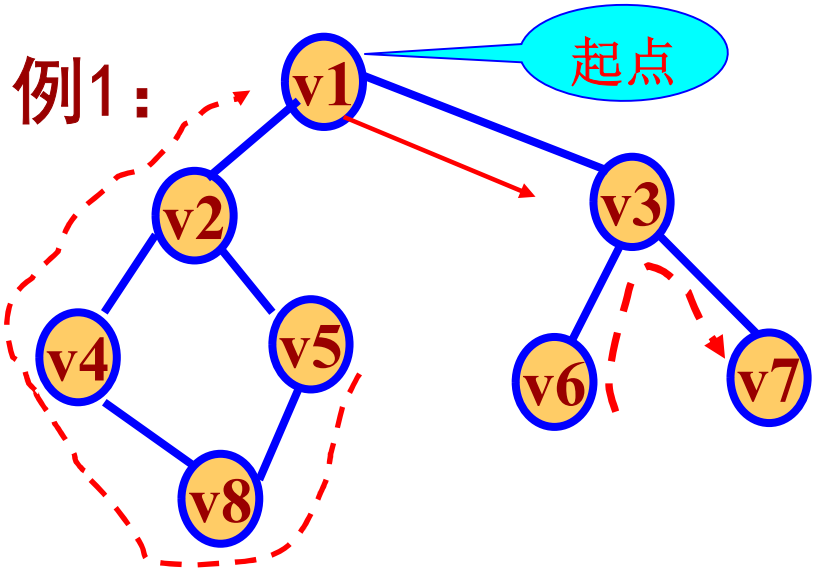
二、广度优先搜索

一、深度优先搜索（ DFS ）

Depth_First Search

基本思想：——仿树的先序遍历过程。 **遍历步骤**

例1：

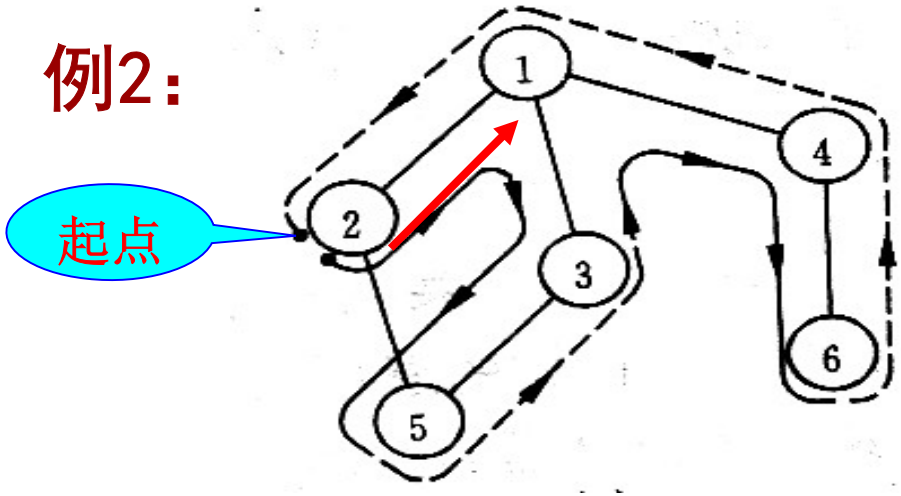


DFS 结果

v1→v2→v4→v8→
v5→v3→v6→v7

应退回到V8，因为V2已有标记

例2：



DFS 结果

v2 → v1 → v3 → v5 →
v4 → v6

应退回到V3，因为V2已有标记

深度优先搜索（遍历）步骤：

简单归纳：

- 访问起始点 v;
- 若v的第1个邻接点没访问过，深度遍历此邻接点；
- 若当前邻接点已访问过，再找v的第2个邻接点重新遍历。

详细归纳：

☛在访问图中某一起始顶点 v 后，由 v 出发，访问它的任一邻接顶点 w_1；

☛再从 w_1 出发，访问与 w_1邻接但还未被访问过的顶点 w_2；

☛然后再从 w_2 出发，进行类似的访问，…

☛如此进行下去，直至到达所有的邻接顶点都被访问过的顶点 u 为止。

☛接着，退回一步，退到前一次刚访问过的顶点，看是否还有其它未被访问的邻接顶点。

如果有，则访问此顶点，之后再从此顶点出发，进行与前述类似的访问；

如果没有，就再退回一步进行搜索。重复上述过程，直到连通图中所有顶点都被访问过为止。

讨论1：计算机如何实现DFS？ ——开辅助数组 *visited* [n]！

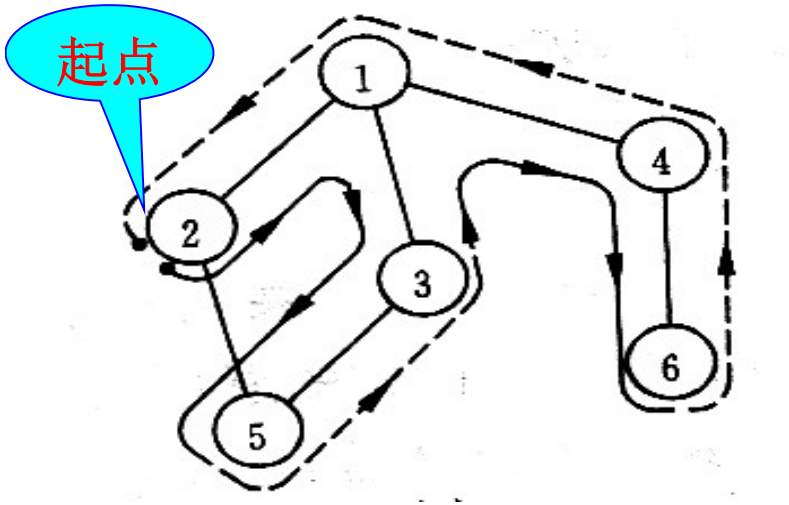
例：

邻接矩阵A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0

辅助数组 *visited* [n]

1	0	0	1	1	1	1	1
2	0	1	1	1	1	1	1
3	0	0	0	1	1	1	1
4	0	0	0	0	0	1	1
5	0	0	0	0	1	1	1
6	0	0	0	0	0	0	1



DFS 结果

v2→v1→v3→v5→v4→v6

请注意逐级回退是递归概念

讨论2： DFS算法如何编程？ ——可以用递归算法！

```
DFS1(A, n, v)     //A[n][n]为邻接矩阵，v为起始顶点(编号)
{ visit（v）;      //访问（例如打印）顶点v
  visited[v]=1;    //访问后立即修改辅助数组标志
 for( j=1; j<=n; j++)    //从v所在行从头搜索邻接点
   if ( A[v, j] && ! visited[j] ) DFS1(A, n, j);
 return;
} // DFS1
```

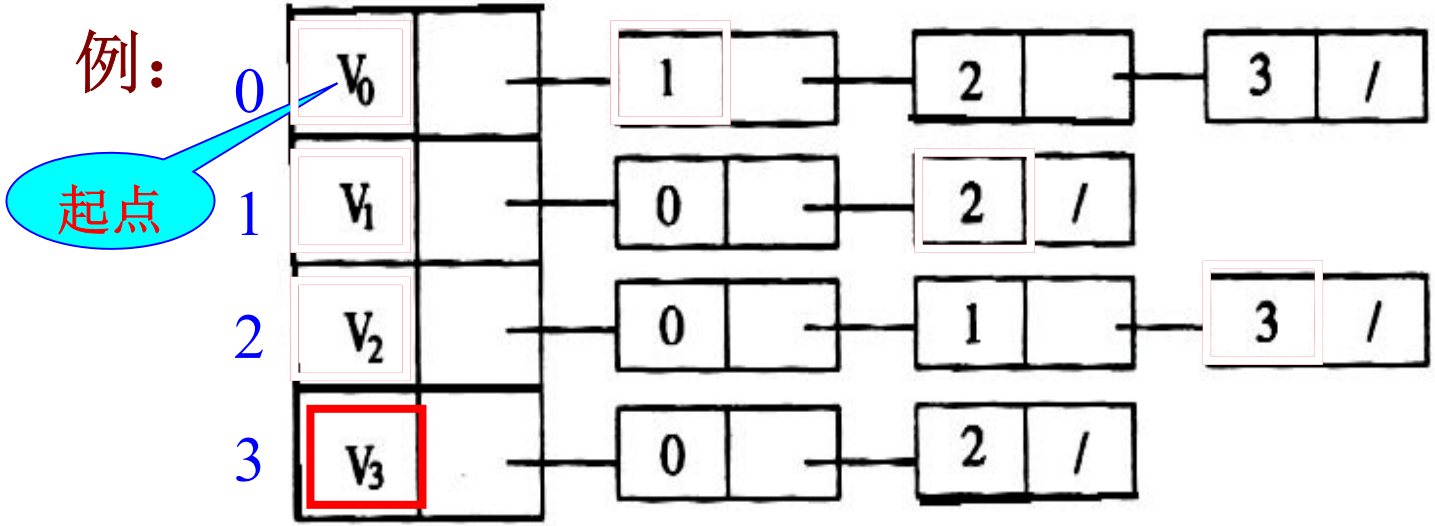
A[v,j] ＝1
有邻接点

visited [n]=0
未访问过

(教材上DFS递归算法见P169)

讨论3：在图的邻接表中如何进行DFS？—照样借用*visited* [n]！

例：



辅助数组 *visited* [n]

0	0	1	1	1	1
1	0	0	1	1	1
2	0	0	0	1	1
3	0	0	0	0	1

DFS 结果

$v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$

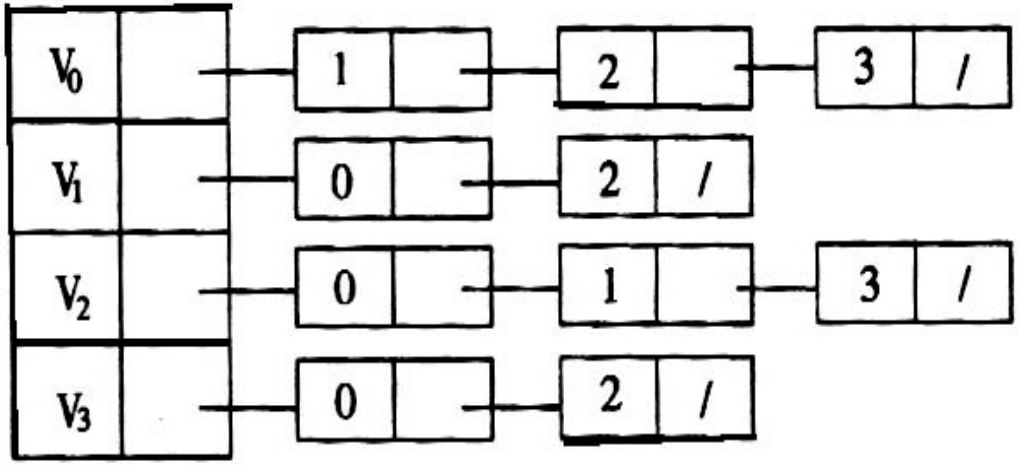
注意：在邻接表中，并非每个链表元素（表结点）都被扫描到,因此遍历速度很快。

讨论4： 邻接表的DFS算法如何编程？

(书上没有，02级同学编，仅供参考)——仍用递归算法

```
Int visited[]; //初始化辅助数组,元素均为0
Void DFS(List,v,p) //设p为v的头指针

{ visit(v); //访问起点
visited[v]=1; //起点已访问,0变1
while(p->link) //当存在起点的第一个邻接点时
  { p=p->link;
     v=p->data;
   if(!visited[v])DFS(List,v,p); //进行递归
   }
return;
}
```



DFS 算法效率分析:

(设图中有 n 个顶点，e 条边)

- 如果用邻接矩阵来表示图，遍历图中每一个顶点都要从头扫描该顶点所在行，因此遍历全部顶点所需的时间为$O(n^2)$。
- 如果用邻接表来表示图，虽然有 $2e$ 个表结点，但只需扫描 e 个结点即可完成遍历，加上访问 n个头结点的时间，因此遍历图的时间复杂度为$O(n+e)$。

结　论:

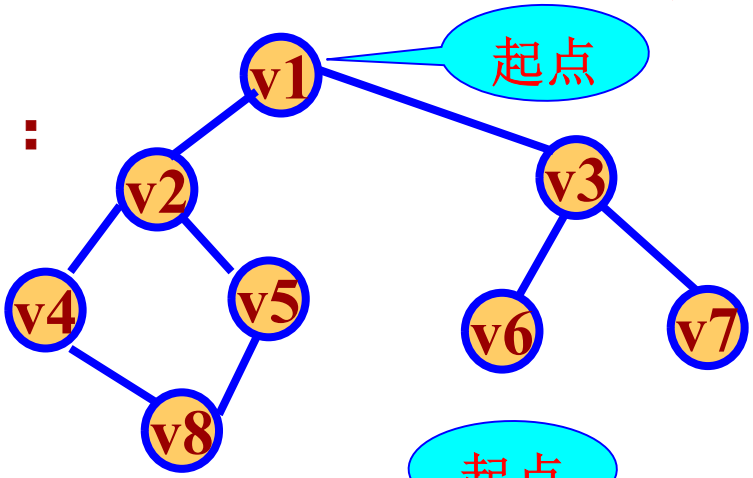
稠密图适于在邻接矩阵上进行深度遍历;
稀疏图适于在邻接表上进行深度遍历。

二、广度优先搜索（ BFS ）

Breadth_First Search

基本思想：——仿树的层次遍历过程。 **遍历步骤**

例1：

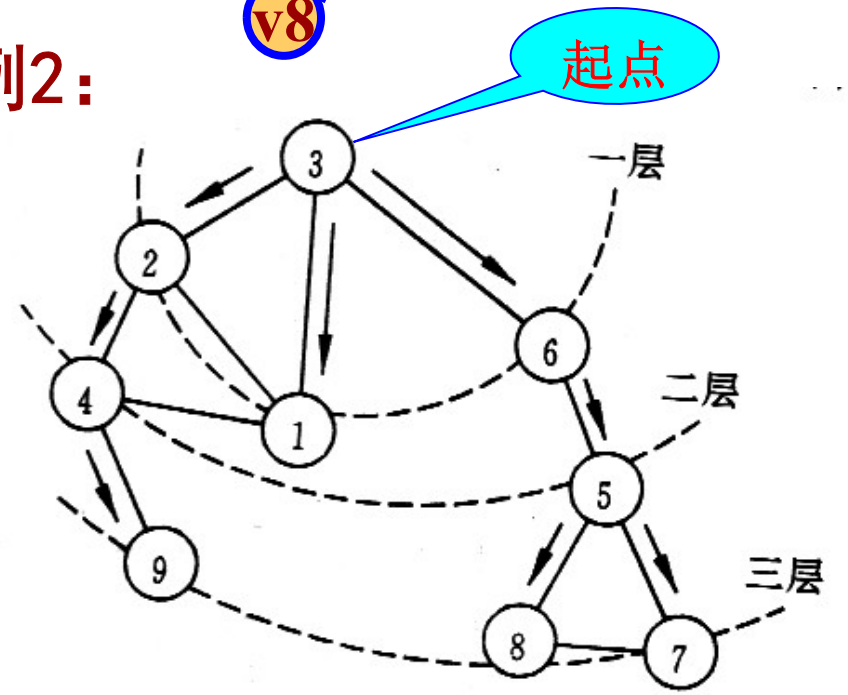


BFS 结果

v1→ v2→ v3 →

v4→v5→v6→v7→v8

例2：



BFS 结果

v3 → v2 → v1 → v6 →

v4 → v5 →v9 → v8 → v7

广度优先搜索（遍历）步骤：

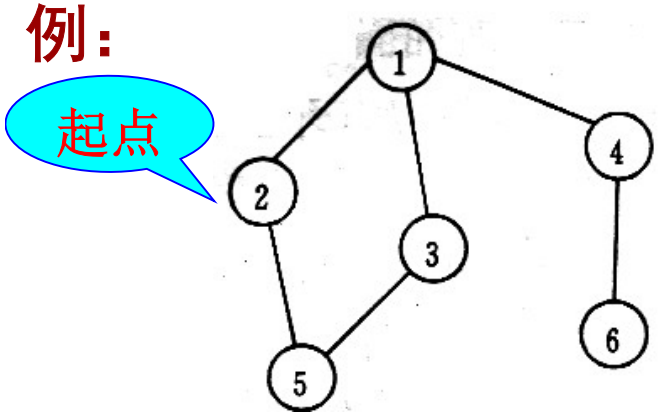
简单归纳：

- 在访问了起始点v之后，依次访问 v的邻接点；
- 然后再依次（顺序）访问这些点（下一层）中未被访问过的邻接点；
- 直到所有顶点都被访问过为止。

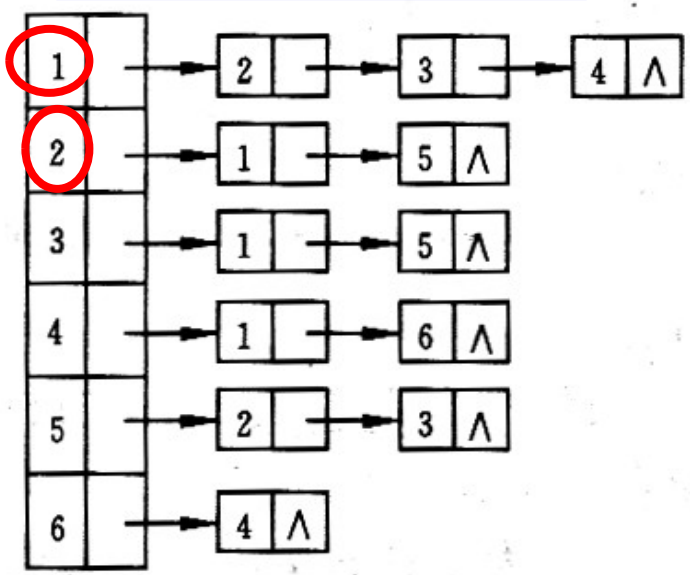
广度优先搜索是一种分层的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。
因此，广度优先搜索不是一个递归的过程，其算法也不是递归的。

讨论1：计算机如何实现BFS？ 宽度优先搜索要借助队列！

例：

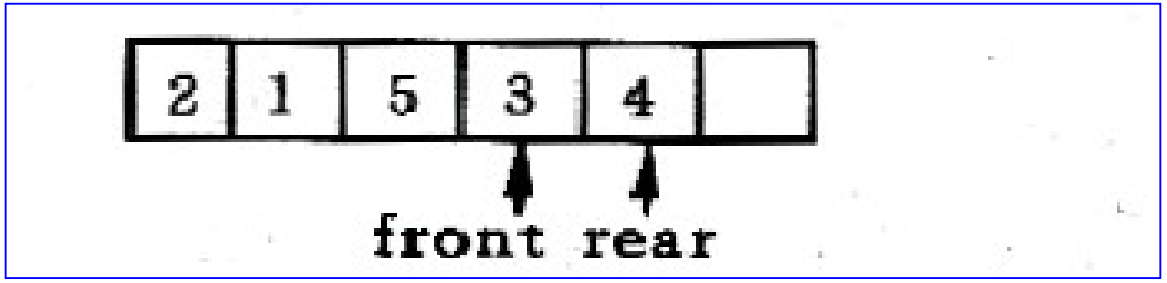


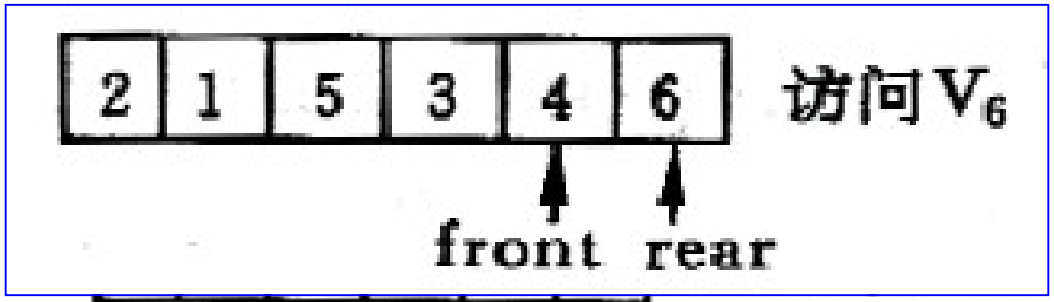
邻接表

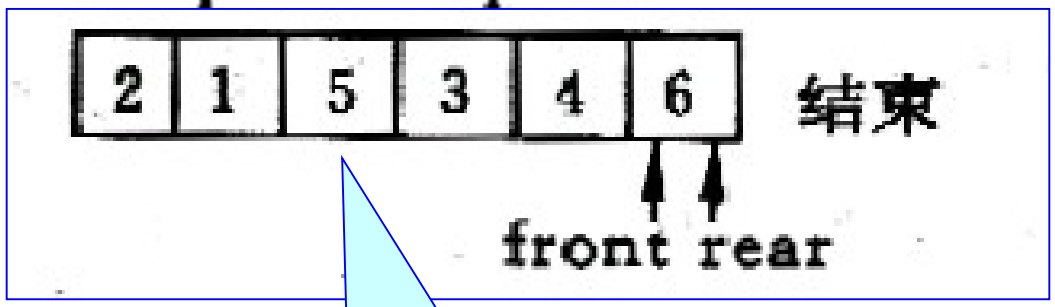


辅助队列

visited [n]仍需要







BFS 遍历结果

讨论2： BFS算法如何编程？ ——层次遍历应当用队列！

```
BFS1(List, n, v) //List为邻接表，v为起点，Q[n]为辅助队列
{ Visit(v); Visited[v]=1;  //访问（例如打印）顶点v并修改标志
   front=n-1;rear=0;        //队列指针初始化
   q[rear]=v;               //起始点入队
 while(rear!=front) //队不空时
  { front=(front+1)%n; v=q[front];   //访问过的顶点出队
    p=List[v].firstarc;                //P指向第1个邻接点
   while(!p)
   { if(! Visited[adjvex(p)] )//未到表尾，且邻接域未访问过，
        {Visit(adjvex(p)); Visited[adjvex(p)]=1;//则先输出再改标记，
             rear=(rear+1)%n; q[rear]= adjvex(p)} //最后再入队
              p=nextarc(p); //指向单链表中下一个邻接点
   }
  }
 return
} // BFS1
```

(教材上BFS算法见P170)

BFS 算法效率分析:

(设图中有 n 个顶点，e 条边)

- 如果使用邻接表来表示图，则BFS循环的总时间代价为 $d_0 + d_1 + \ldots + d_{n-1} = O(n+e)$，其中的 d_i 是顶点 i 的度。
- 如果使用邻接矩阵，则BFS对于每一个被访问到的顶点，都要循环检测矩阵中的整整一行（n 个元素），总的时间代价为$O(n^2)$。

DFS与BFS之比较:

- 空间复杂度相同，都是O(n)(借用堆栈或队列装n个顶点)；
- 时间复杂度只与存储结构（邻接矩阵或邻接表）有关，而与搜索路径无关。

7.4 图的其他运算

1. 求图的生成树
2. 求最小生成树
3. 求最短路径
4. 求关节点和重连通分量（略）
5. 拓扑排序
6. 求关键路径

1.求图的生成树（或生成森林）

生成树：是一个极小连通子图，它含有图中全部顶点，但只有n-1条边。

生成森林：由若干棵生成树组成，含全部顶点，但构成这些树的边是最少的。

思考1：若对连通图进行遍历，得到的是什么？

——得到的将是一个极小连通子图，即图的生成树！

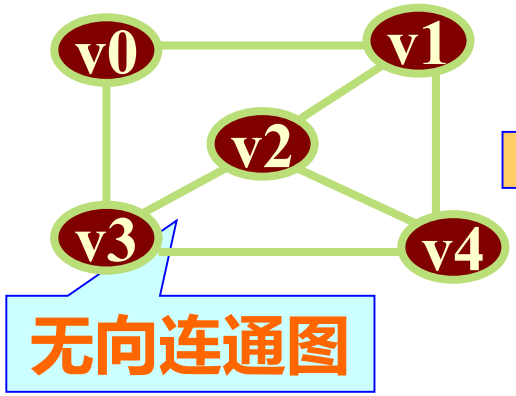
由深度优先搜索得到的生成树，称为深度优先搜索生成树。

由广度优先搜索得到的生成树，称为广度优先搜索生成树。

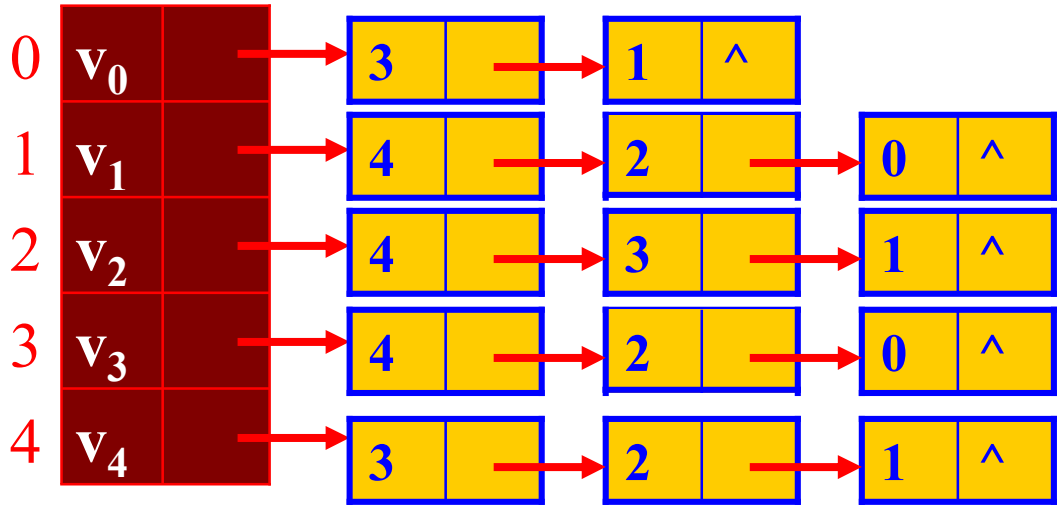
思考2：若对非连通图进行遍历，得到的是什么？

—— 得到的将是各连通分量的生成树，即图的生成森林！

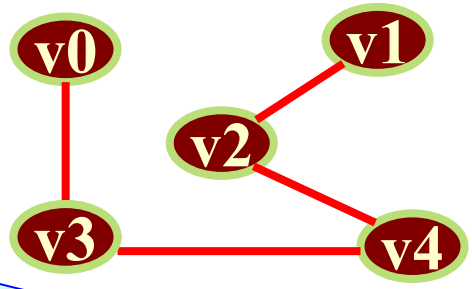
例1：画出下图的生成树



邻接表

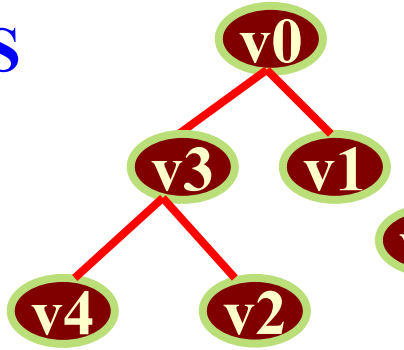


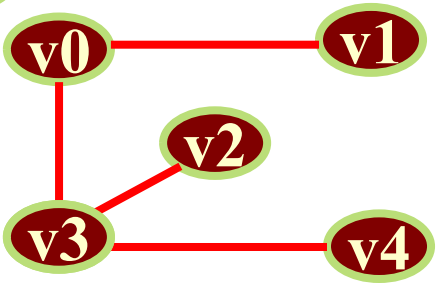
DFS 生成树



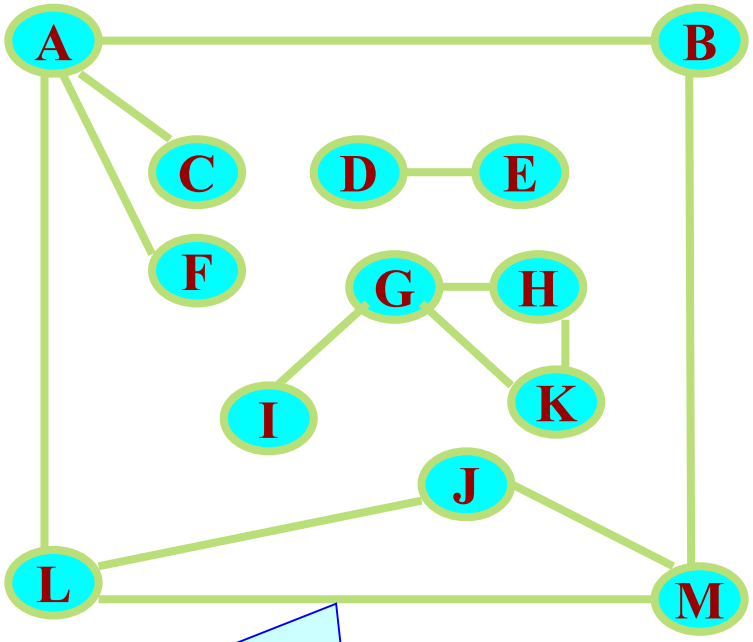
本例无回溯

BFS 生成树





例2：画出下图的生成森林（或极小连通子图）



生成森林的定义（对有向或无向图均适用）：是若干棵生成树的集合，含全部顶点，但构成这些树的边或弧是最少的。

求解步骤：

Step1:先求出邻接矩阵或邻接表；

Step2:写出DFS或BFS结果序列；

Step3:画出对应子图或生成森林。

这是一个无向非连通图
(参见教材P170-171例)

其实由邻接矩阵或邻接表也能直接画出生成森林

下面选用邻接表方式来求深度优先搜索生成森林

先写出邻接表（或邻接矩阵）：

0	A	→ 1 → 2 → 5 → 11 ^
1	B	→ 0 → 12 ^
2	C	→ 0 ^
3	D	→ 4 ^
4	E	→ 3 ^
5	F	→ 0 ^
6	G	→ 7 → 8 → 10 ^
7	H	→ 6 → 10 ^
8	I	→ 6 ^
9	J	→ 11 → 12 ^
10	K	→ 6 → 7 ^
11	L	→ 0 → 9 → 12 ^
12	M	→ 1 → 9 → 11 ^

最小连通！

再写出DFS结果

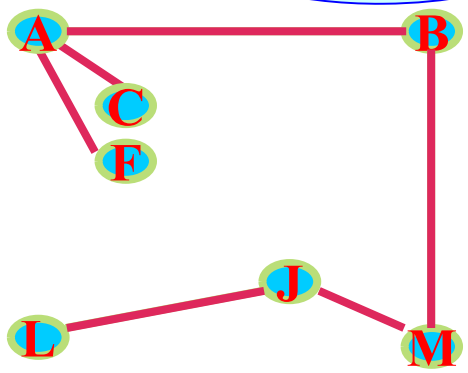
ABMJLCF

DE

GHKI

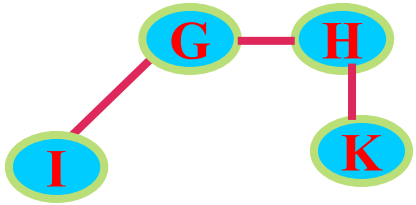
思考：怎样找3个根?

子图1：

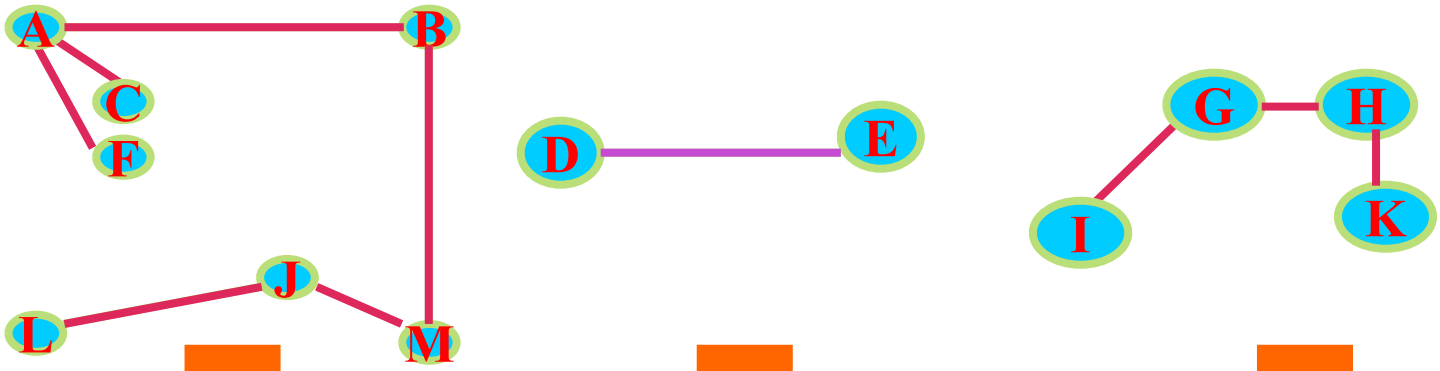


子图2：

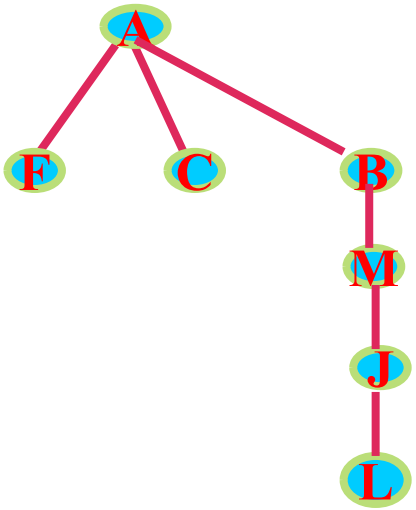
子图3：

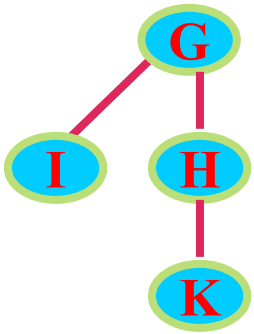


子图
(或连通分量)



生成森林





2. 求最小生成树

目标：**在网络的多个生成树中，寻找一个各边权值之和最小的生成树。**

即有权图

首先明确：

- **使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。**
- **按照生成树的定义，*n* 个顶点的连通网络的生成树肯定有 *n* 个顶点和仅仅*n*-1 条边。**

构造最小生成树的准则

- **必须只使用该网络中的边来构造最小生成树；**
- **必须使用且仅使用*n*-1条边来联结网络中的*n*个顶点；**
- **不能使用产生回路的边。**

典型用途：

欲在n个城市间建立通信网，则n个城市应铺n-1条线路；但因为每条线路都会有对应的经济成本，而n个城市可能有n(n-1)/2 条线路，那么，如何选择*n–1*条线路使总费用最少？

先建立数学模型：

顶点———*表示城市，有n个；*

边————*表示线路，有n–1条；*

边的权值—*表示线路的经济代价；*

连通网——*表示n个城市间的通信网。*

显然此连通网是一棵生成树！

问题抽象： n个顶点的生成树很多，需要从中选一棵代价最小的生成树，即该树各边的代价之和最小。此树便称为最小生成树MST。

Minimum cost Spanning Tree

讨论：如何求得最小生成树？

最小生成树（MST）的性质如下：

若U集是V的一个非空子集，若(u_0, v_0)是一条最小权值的边，其中$u_0 \in U$，$v_0 \in V-U$；则：(u_0, v_0)必在最小生成树上。

设想一下：先把权值最小的边归入生成树内，逐个递增，舍去回路边，则得到的很可能就是最小生成树！

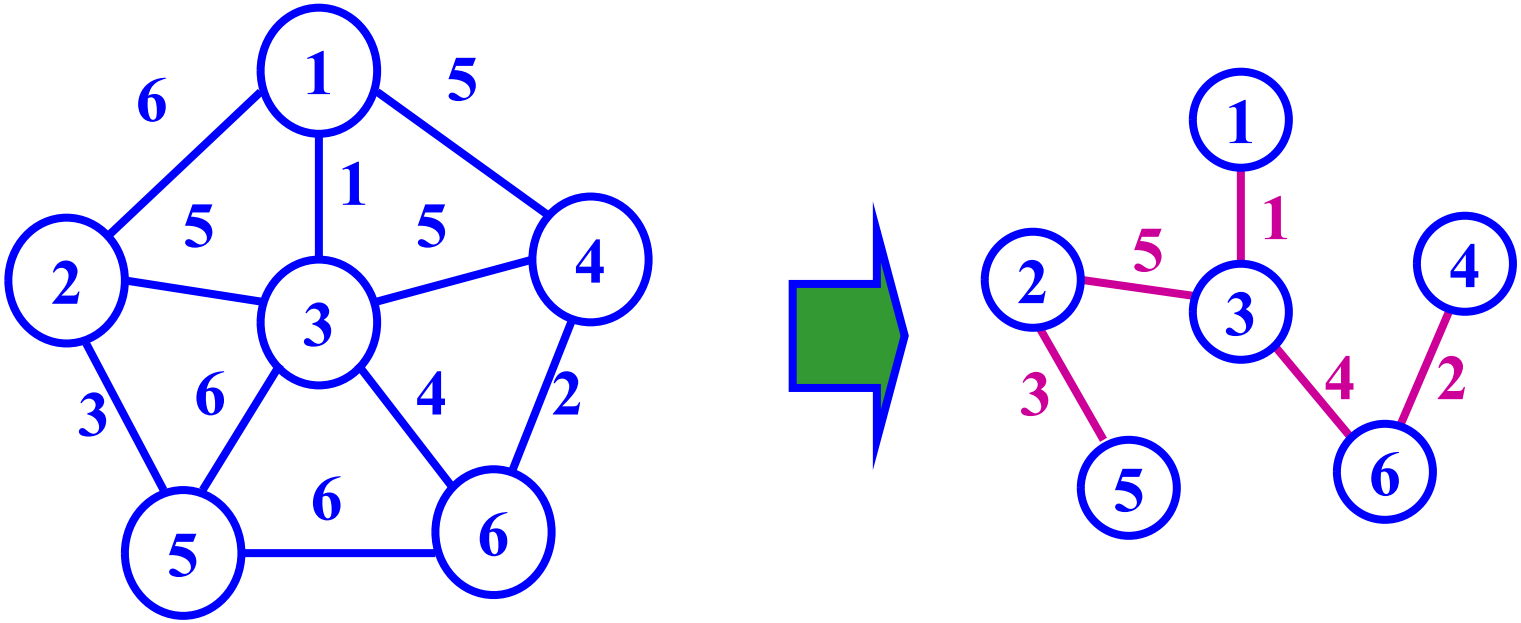
求MST有多种算法，但最常用的是以下两种：

- Kruskal（克鲁斯卡尔）算法
- Prim（普里姆）算法

Kruskal算法特点：将边归并，适于求稀疏网的最小生成树。

Prime算法特点: 将顶点归并，与边数无关，适于稠密网。

Kruskal算法示例：对边操作，归并边

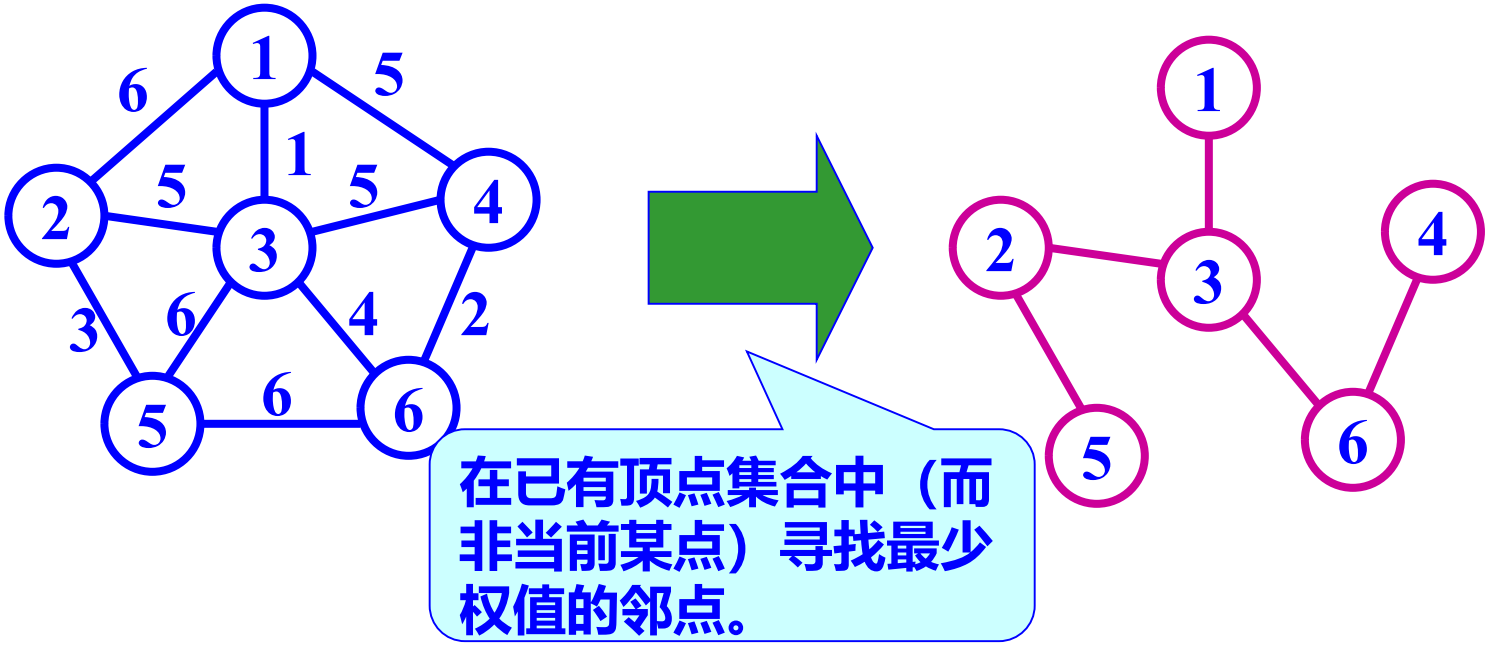


Kruskal算法效率分析：

Kruskal算法的时间效率 = O（$e\log_2 e$）

Kruskal算法是归并边，适用于稀疏图（用邻接表）

普利姆（Prim）算法示例： 归并顶点

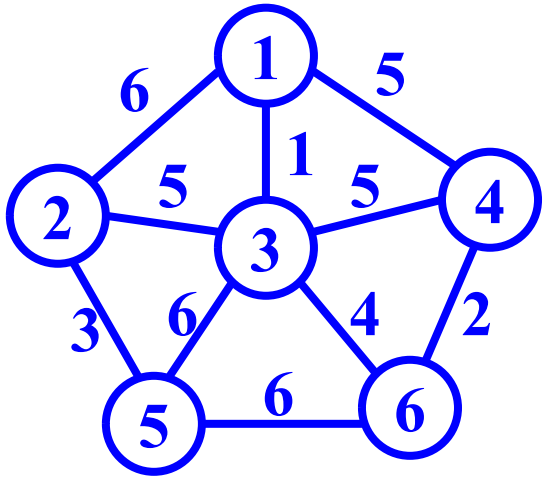


Prim算法效率分析：

Prim算法的时间效率 = O（n^2）

Prim算法是归并顶点，适用于稠密网。

普利姆（Prim）算法示例(PP175)： 算法7.9




```
//记录从顶点集U到V-U的代价最小的边的辅助数组
struct{
  VertexType     adjvex;
  VRTYPE         lowcost;
}closedge[MAX_VERTEX_NUM];

void MiniSpanTree_PRIM(MGRAPH G, VertexTYPE u){
// 用Prim算法从第u个顶点出发构造网G的最小生成树T，
输出T的各条边
   k = LocateVex(G,u);
   for(j=0; j<G.vexnum; ++j)
      if(j!=k) closedge[j] =  {u, G.arcs[k][j].adj};
   closedge[k].lowcost = 0;    // 初始U = {u}
   for(i=1;i<G.vexnum;++i){    //选择其余G.vexnum -1 个顶点
      k = minimum(closedge);  //求出T的下一个节点；第k顶点
      printf(closege[k].adjvex,G.vexs[k]);   //输出生成树的边
      closedge[k].lowcost = 0;    //第k顶点并入U集
      for(j=0;j<G.vexnum;++j)
         if(G.arcs[k][j].adj<closedge[j].lowcost)   //新顶点并入U后重新选择最小边
            closedge[j] = {G.vexs[k],G.arcs[k][j].adj};
    }
}
```